Directed Adaptive Stimulus Evolution Coverage-Directed Evolutionary Stimulus Generation for RISC-V RTL Verification

Ankush Kumar Jaiswal, Aarav Sharma, Garvit Maheshwari, Jean Jenifer Nesam

School of Electronics Engineering (SENSE), Vellore Institute of Technology, Chennai, India

accellera SYSTEMS INITIATIVE

2026 DVCON INDIA



Ankush Kumar Jaiswal is a final-year B.Tech student in **Electronics and Computer Engineering** at VIT Chennai. His research interests include digital design verification, RISC-V processor architectures, FPGA-based systems, and AI-assisted Electronic Design Automation (EDA). His work focuses on developing intelligent verification methodologies using SystemVerilog, UVM, and machine learning to improve functional coverage and verification efficiency. He is passionate about advancing semiconductor design and verification through innovative hardware and AI-driven solutions.

© Accellera Systems Initiative



Aarav Sharma is a fourth-year B.Tech. student in Electronics and Computer Engineering at Vellore Institute of Technology (VIT), Chennai. His academic interests include VLSI design, computer architecture, digital system design, embedded systems, and Electronic Design Automation (EDA). He is passionate about developing efficient hardware systems and exploring the application of machine learning techniques to improve hardware performance and design workflows. His research focuses on intelligent optimization methods for next-generation computing systems, with an emphasis on bridging the gap between hardware design and AI-driven automation. He aspires to pursue advanced research in semiconductor technologies and EDA, contributing to the development of high-performance and energy-efficient integrated circuits

© Accellera Systems Initiative



Garvit Maheshwari is a final-year B.Tech student in Electronics Engineering (VLSI Design and Technology) at VIT Chennai. His areas of interest include digital design, RTL design and verification, computer architecture, and ASIC design. He has worked on academic and research projects involving Verilog/SystemVerilog, RISC-V processor design, and digital hardware verification. He has also gained experience with EDA tools and digital design workflows through coursework and project work. His current work focuses on VLSI design and verification methodologies, with an interest in developing reliable and efficient digital integrated circuits.

© Accellera Systems Initiative

Table of Contents

- Part I (01–08) — Introduction & Motivation
- Part II (09–13) — Background & Related Work
- Part III (14–20) — DASE Framework Architecture
- Part IV (21–30) — Fitness, Evolution, Algorithms & Examples
- Part V (31–34) — Evaluation Methodology
- Part VI (35–42) — Experimental Results
- Part VII (43) — Conclusion & Future Work

© Accellera Systems Initiative

What is RTL Verification?

- **Register Transfer Level (RTL) Verification** is the process of validating that a hardware design written in Verilog/SystemVerilog behaves exactly according to its specification **before fabrication**. It ensures functional correctness, detects design bugs early, and significantly reduces the risk of costly silicon re-spins.

© Accellera Systems Initiative

The Verification Bottleneck

Modern System-on-Chip designs have grown so complex that functional and structural verification now consumes more than half of the entire design effort especially for processor cores where deep microarchitectural corner cases must be validated.

- >50% of design effort spent on verification
- RISC-V open ISA shifts verification burden to design teams
- Constrained Random Verification (CRV) — industry standard, probabilistic
- Deep microarchitectural corners need many regression cycles

© Accellera Systems Initiative

Why Verification Is Hard

- Explosion of state space
  - Deep pipelines, multi-issue, out-of-order execution, complex memory hierarchies
  - Every legal state transition must be verified — combinatorial growth
- The RISC-V inflection
  - Open ISA shifts verification burden onto design teams
  - No proprietary verification IP → efficient methodology critical

© Accellera Systems Initiative

CRV: Industry Standard, With Limits

Constrained Random Verification generates large volumes of legal instruction sequences to explore the state space — but its probabilistic nature makes it inefficient at reaching deep corner cases.

- Long RAW dependency chains rarely produced
- Load-after-store hazards need precise sequencing
- Complex branch interactions need directed stimulus
- Consequence: manual directed tests, long regression cycles

© Accellera Systems Initiative

The Corner Cases That Matter

Structural bugs in processors hide not in individual instruction execution, but in *microarchitectural interactions* — exactly where random stimulus struggles to reach.

- READ-AFTER-WRITE DEPENDENCY CHAIN

  Cyc 1 add x1, x2, x3

  Cyc 2 sub x4, x1, x5← **RAW hazard**[forwarded]

  Cyc 3 or x6, x4, x7← **RAW hazard**

- LOAD-AFTER-STORE MEMORY HAZARD

  Cyc 1 sw x1, 0(x2)

  Cyc 2 lw x3, 0(x2)← **load-after-store hazard**

© Accellera Systems Initiative

What's Missing (Research Gap)

- Existing approaches: heavy computational overhead
  - ML-guided stimulus — needs training data
  - RL approaches — need GPU acceleration
  - Hardware fuzzing — demands FPGA emulation
- What we need:
  - lightweight, coverage-aware, structurally directed generator

© Accellera Systems Initiative

The Landscape of Verification Stimulus

- 2003 — Bayesian Network CDG (Fine & Ziv)
- 2018 — RFUZZ on FPGAs (Laeufer et al.)
- 2022 — Genetic Algorithm Verification (Danciu & Dinu)
- 2022 — MicroGP (Squillero)
- 2023 — MorFuzz (Xu et al.)
- 2024+ — ML / RL / Lyra

© Accellera Systems Initiative

Closing the Feedback Loop

- Coverage-Directed Test Generation (CDG) automates the feedback loop
- Directs tests toward uncovered points, not probability
- Loop: Generate Stimulus → Simulate DUT → Collect Coverage → Direct Next Stimulus



© Accellera Systems Initiative

Genetic Algorithms Meet Hardware

Evolutionary methods treat *instruction sequences as genomes* — evolved through mutation, crossover, and selection against coverage metrics.

- Instruction sequences treated as genomes
- Evolved via mutation, crossover, selection against coverage metrics
- Genetic operators:

  Selection — fittest sequences reproduce

  Crossover — recombine promising patterns

  Mutation — introduce new instruction variants

  Replacement — evolve population across generations

© Accellera Systems Initiative

Powerful, But Heavy (ML & RL)

- ML-guided stimulus generation (Gogri et al.) — needs labeled training data
- RL-based approach (Gadde et al.) — GPU-intensive
- Lyra framework (Huo et al.) — FPGA-accelerated fuzzing
- Common drawback: significant computational overhead

© Accellera Systems Initiative

Python & cocotb: The Enabler

Python-based verification frameworks such as *cocotb* made it feasible to integrate algorithmic stimulus generation directly into RTL simulation — bridging software optimization with hardware simulation.

- cocotb bridges Python ecosystem with RTL simulation
- Combines Python ML/optimization tools with RTL sims
- Reuses existing SystemVerilog verification IP
- Enables closed-loop evolutionary frameworks

© Accellera Systems Initiative

What This Paper Delivers

- Lightweight evolutionary framework for structural coverage closure in RISC-V processors
- Multi-objective structural fitness function targeting microarchitectural hazards and pipeline behavior
- Closed-loop evolutionary verification architecture integrated with Python and cocotb

© Accellera Systems Initiative

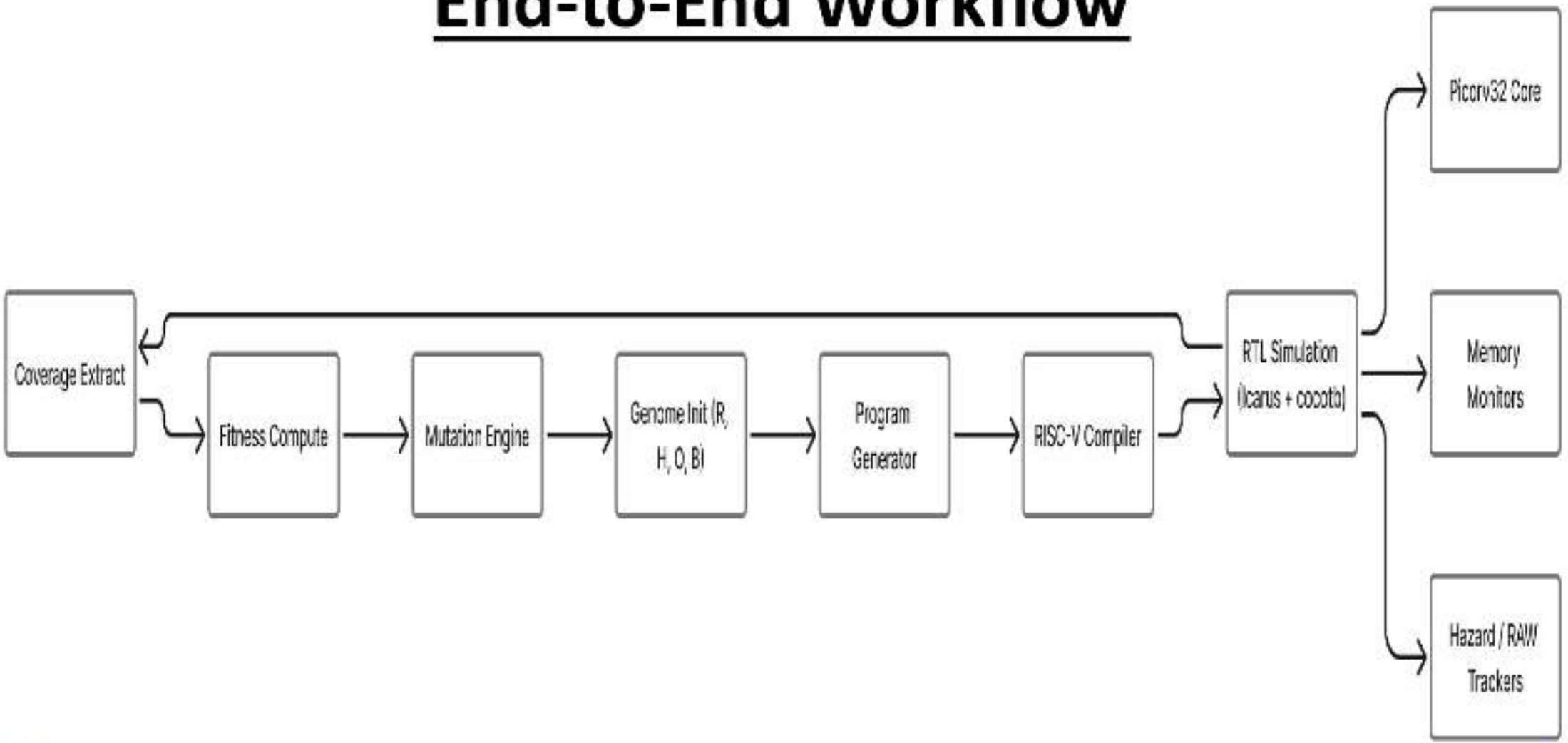
DASE: The Idea

The DASE framework builds upon these concepts by combining **evolutionary stimulus generation** with **structural coverage-driven fitness evaluation** to improve coverage closure efficiency in processor verification environments.

- Closed-loop evolutionary stimulus generation system
- Integrates genetic algorithms with structural coverage feedback
- Iteratively evolves RISC-V assembly programs
- Python Orchestration Engine — genome init, mutation, fitness eval, selection
- RTL Simulation Environment — toolchain compile, RTL execution, metric extraction, coverage feedback

© Accellera Systems Initiative

End-to-End Workflow



Initialization & Generation

- Genome Initialization
  - Population of N random RISC-V instruction sequences
  - Genome = ordered list of (opcode, operands, immediates)

```
population = []
for i in range(N):
genome = [random_instr() for _ in range(L)]
population.append(genone)
```

- Assembly Program Construction
  - Genome converted to valid RISC-V assembly with labels, register allocation, termination sequence

```
def genome_to_asm(genome):
lines = [".text", "_start:"]
for g in genome: lines.append(g.asm())
lines.append("ecall")
return "\n".join(lines)
```


© Accellera Systems Initiative

Compile, Simulate, Extract

- RISC-V Toolchain Compilation
  - Assembly compiled via standard RISC-V GCC toolchain to hex, loaded into instruction memory

```
riscv32-unknown-elf-as -march=rv32i \
-o prog.o prog.s
riscv32-unknown-elf-ld -T linker.ld \
-o prog.elf prog.o
riscv32-unknown-elf-objcopy -O binary \
prog.elf prog.bin
```

- RTL Simulation & Metric Extraction
  - Binary runs on PicoRV32 via cocotb
  - Structural metrics extracted: RAW hazards, load-after-store hazards, opcode counts, branch polarity

```
@cocotb.test()
async def run_dase(dut):
await load_program(dut, "prog.bin")
await drive_clock(dut, cycles=N)
metrics = extract_structural(dut)
metrics["fitness"] = compute_fitness(metrics)
```


© Accellera Systems Initiative

Fitness, Selection, Evolution

- Fitness Evaluation
  - Multi-objective score from RAW depth, memory hazards, opcode diversity, branch polarity

```
def compute_fitness(m):
F = w1*m["RAW"] + w2*m["HAZ"]
+ w3*m["OPC"] + w4*m["BR"]
+ w5*balance(m)
return F
```

- Selection · Crossover · Mutation
  - Fitness-proportional selection; crossover recombines parent segments; mutation adds controlled perturbation

```
parents = roulette_select(pop, k=2)
child = crossover(parents[0], parents[1])
if random() < p_mut:
child = mutate(child)
next_gen.append(child)
```


© Accellera Systems Initiative

Stressing the Pipeline

STAGE 01	STAGE 02	STAGE 03	STAGE 04	STAGE 05
Fetch	**Decode**	**Execute**	**Memory**	**Writeback**
Instruction memory access, PC update	Opcode decode, register read, hazard detection	ALU ops, forwarding, branch evaluation	Load/store, memory hazards, interface stalls	Register file update, dependency resolution

© Accellera Systems Initiative

Multi-Objective Structural Fitness

Unlike traditional functional coverage, our fitness function targets ***microarchitectural stress conditions*** — the exact corner cases where random stimulus fails.

- $F = w_1 \cdot R + w_2 \cdot H + w_3 \cdot O + w_4 \cdot B + w_5 \cdot \text{Balance}$
- R — RAW depth: long dependency chains stressing forwarding logic
- H — Hazards: load-after-store memory hazards
- O — Opcodes: diversity prevents overfitting
- B — Branch: both taken & not-taken paths executed
- Balance — prevents maximizing one metric alone

© Accellera Systems Initiative

R — Read-After-Write Depth

- Promotes long dependency chains stressing forwarding logic & hazard detection
- Each dependent instruction forces forward or stall
- Forwarding paths are a common source of subtle bugs
- Random stimulus rarely exceeds 2–3 chain length; evolution pushes deeper
- DEPENDENCY CHAIN DEPTH: 4 (HIGH-FITNESS PATTERN)

Cyc 1	add x1, x2, x3	**chain start**
Cyc 2	sub x4, x1, x5	**depends on x1**
Cyc 3	or x6, x4, x7	**depends on x4**

© Accellera Systems Initiative

H — Load-After-Store Hazards

- Stresses memory interface and load/store interactions
- Targets a load immediately following a store to the same address
- Random generation hits these only by chance; evolution drives systematically

▸ LOAD-AFTER-STORE HAZARD PATTERNS

Pattern A	sw x1, 0(x2) → lw x3, 0(x2)	**same-address hazard**
Pattern B	sw x1, 4(x2) → lw x3, 4(x2)	**offset hazard**

© Accellera Systems Initiative

O · B — Diversity & Balance

- O — Opcode diversity: prevents overfitting to a single instruction type
- B — Branch polarity: ensures both taken/not-taken paths execute
- The **balance term (w_5)** prevents the algorithm from maximizing one metric while ignoring others. It penalizes extreme imbalance — e.g., maximizing RAW depth while losing all branch diversity — encouraging well-rounded structural coverage.

© Accellera Systems Initiative

The Evolution Strategy

- Selection probability proportional to fitness (roulette-wheel):
  $P_i = F_i / \sum_{j=1}^{N} F_j$
- Fitness-proportional selection
- Single/multi-point crossover
- Mutation: opcode swap, operand change, immediate randomization
- Evolves over generations: deeper chains, more hazards, broader opcodes, higher structural coverage

© Accellera Systems Initiative

Algorithm 1: DASE Main Loop

- Input:

  Population size N

  Sequence length L

  Number of generations G

  Fitness weights w1...w5

  Mutation rate pm

  Crossover rate pc

Output:

  Evolved population P*

Phase I: Initialization

1: P ← InitializePopulation(N, L)

2: for each genome gi ∈ P do

3: asm ← GenomeToAssembly(gi)

4: bin ← Compile(asm, RISC-V Toolchain)

5: metrics ← Simulate(bin, PicoRV32 RTL)

6: Fi ← ComputeFitness(metrics, w1...w5)

7: end for

© Accellera Systems Initiative

Algorithm 1: DASE Main Loop(continues)

```
Phase II: Evolution
8:      for generation = 1 to G do
9:          Pnew ← Elitism(P, k = 2)
10:         while |Pnew| < N do
11:             p1 ← RouletteSelect(P, F)
12:             p2 ← RouletteSelect(P, F)
13:             if rand() < pc then
14:                 child ← Crossover(p1, p2)
15:             else
16:                 child ← Copy(p1)
17:             end if

18:             if rand() < pm then
19:                 child ← Mutate(child)
20:             end if
```

Algorithm 1: DASE Main Loop(continues)

```
21:                 asm ← GenomeToAssembly(child)
22:                 bin ← Compile(asm, RISC-V Toolchain)
23:                 metrics ← Simulate(bin, PicoRV32 RTL)
24:                 Fc ← ComputeFitness(metrics, w1...w5)
25:                 Insert (child, Fc) into Pnew
26:         end while
27:         P ← Pnew
28: end for
29: return P*
```


© Accellera Systems Initiative

Algorithm 2: Structural Fitness

Input:

Structural metrics

m = {RAW, HAZ, OPC, BR}

Fitness weights w1...w5

Output:

Fitness score F ∈ [0,1]

1: R ← min(RAWdepth, Rmax) / Rmax

2: H ← min(HAZcount, Hmax) / Hmax

3: O ← Number of unique opcodes / Total opcode universe

4: B ← min(BranchesTaken, BranchesNotTaken) / max(BranchesTaken,BranchesNotTaken + ε)

© Accellera Systems Initiative

Algorithm 2: Structural Fitness(continue)

5: $\mu \leftarrow (R + H + O + B) / 4$

6: $\sigma \leftarrow \text{sqrt}(((R - \mu)^2 + (H - \mu)^2 + (O - \mu)^2 + (B - \mu)^2) / 4)$

7: $\text{Balance} \leftarrow 1 - \sigma$

8: $F \leftarrow w1R + w2H + w3O + w4B + w5\text{Balance}$

9: $F \leftarrow \text{clip}(F, 0, 1)$

10: return F

© Accellera Systems Initiative

Self-Tuning Extension: The Weight Problem

- The fitness weights [0.40, 0.25, 0.15, 0.10, 0.10] were originally hardcoded
-
- Problem:
  - Hand-tuned weights are hard to defend
  - Different CPUs may need different weights
  - No principled way to choose ceilings [80, 15, 8]
-
- Solution (v1.2): CMA-ES self-tuning
  - Outer loop evolves the 5 weights + 3 ceilings
  - Meta-fitness = Cohen's d (effect size)
  - 8-parameter continuous optimization
  - No hardcoded values — all evolved

Layer 2: CMA-ES Weight Tuning

- Bilevel optimization: outer CMA-ES + inner GA (roulette + crossover)
-
- CMA-ES searches 8 dimensions:
  - 5 weights: softmax → simplex, clipped to [0.05, 0.60]
  - 3 ceilings: sigmoid → [20,150], [5,40], [4,16]
-
- Meta-fitness = Cohen's d (NOT mean DASE fitness):
  - d = (mean_DASE - mean_RAND) / pooled_std
  - Invariant to gaming (lowering ceilings inflates both equally)
  - Forces search to maximize DASE-vs-Random separation
-
- Configuration: max_fevals=100, meta_seeds=6, σ■=0.6
  - Uses Hansen's cma package v4.4.4

How CMA-ES Works

- CMA-ES = Covariance Matrix Adaptation Evolution Strategy
-
- Algorithm:
  - 1. Sample λ offspring from multivariate Gaussian
  - 2. Evaluate each (run inner GA with proposed weights)
  - 3. Rank offspring by meta-fitness (Cohen's d)
  - 4. Update mean and covariance based on successful offspring
  - 5. Repeat until convergence
-
- Key property: covariance adaptation
  - Learns which directions in weight space correlate with good fitness
  - Elongates search along productive directions
  - Contracts along unproductive directions
  - State-of-the-art for continuous parameter optimization

Example: Initialization & Simulation

Let's trace a simplified example. We initialize a population of three short instruction sequences (Genomes A, B, C) and simulate them on the PicoRV32 RTL.

Genome A

```
add  x1, x2, x3
sub  x4, x1, x5  # RAW
lw   x6, 0(x7)
sw   x6, 0(x7)   # HAZ
beq  x1, x4, L1
```

Output:
RAW: 2
HAZ: 1
OPC: 4

Genome B

```
lw   x1, 0(x2)
sw   x3, 0(x2)   # HAZ
add  x4, x5, x6
beq  x4, x5, L2
```

Output:
RAW: 1
HAZ: 1
OPC: 4

Genome C

```
add  x1, x2, x3
add  x4, x1, x5  # RAW
add  x6, x4, x7  # RAW
add  x8, x6, x9  # RAW
```

Output:
RAW: 4
HAZ: 0
OPC: 1

© Accellera Systems Initiative

Example: Fitness Evaluation

We apply the multi-objective fitness function to Genome A. Raw metrics are normalized to [0,1] and combined with weights.

$F = w_1 \cdot R + w_2 \cdot H + w_3 \cdot O + w_4 \cdot B + w_5 \cdot \text{Balance}$

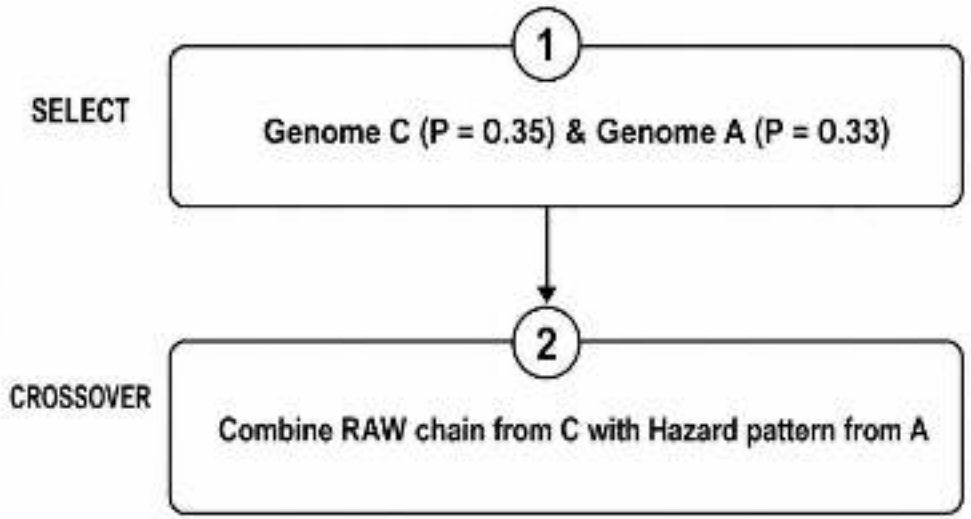
- Metrics (A): R=2 → 0.50, H=1 → 0.33, O=4 → 0.80, B=1 → 1.0
- Balance (A): $\mu = 0.66$, $\sigma = 0.24$ → Balance = 0.76
- Weights: w_1=0.3, w_2=0.25, w_3=0.2, w_4=0.15, w_5=0.1
- **Fitness F_A: 0.3(0.5) + 0.25(0.33) + 0.2(0.8) + 0.15(1.0) + 0.1(0.76) =0.619**

© Accellera Systems Initiative

Example: Selection & Evolution

Using fitness-proportional selection, we calculate the probability of each genome being chosen for reproduction. $P_i = F_i / \sum_{j=1}^{N} F_j$

- Sum Fitness: F_{total} = 0.619 + 0.585 + 0.660 = 1.864
- Probabilities: P_A = 0.33, P_B = 0.31, P_C = 0.35



Evaluation Strategy

Multiple experimental runs across different random seeds ensure *statistical validity and reproducibility* — avoiding bias from favorable initialization.

- Mean fitness: $P_i = F_i / \sum_{j=1}^{N} F_j$
- Statistical rigor:
  - confidence intervals computed via standard methods
  - Welch's t-test
  - Cohen's d over multiple seeds per condition
- Metrics:
  - mean structural fitness
  - instruction diversity
  - register usage
  - pipeline coverage,
  - throughput (IPS)

© Accellera Systems Initiative

Why Statistics Matter

Stochastic optimization algorithms are vulnerable to lucky initializations. Statistical validation proves that observed improvements are *real, not random*.

- **Welch's t-test** — compares means of unequal-variance samples

  P-value threshold: $\alpha = 0.05$;

  result $p = 0.000132$

- **Cohen's d** — measures magnitude of difference

  0.2 small , 0.5 medium , 0.8+ large;

  result $d = 2.0$ (very large)

© Accellera Systems Initiative

The Test Bench

OUT

PicoRV32

Open-source RISC-V RV32I core

FRAMEWORK

Python + cocotb

Closed-loop evolutionary harness

TOOLCHAIN

riscv-gcc

RV32I assembly → binary

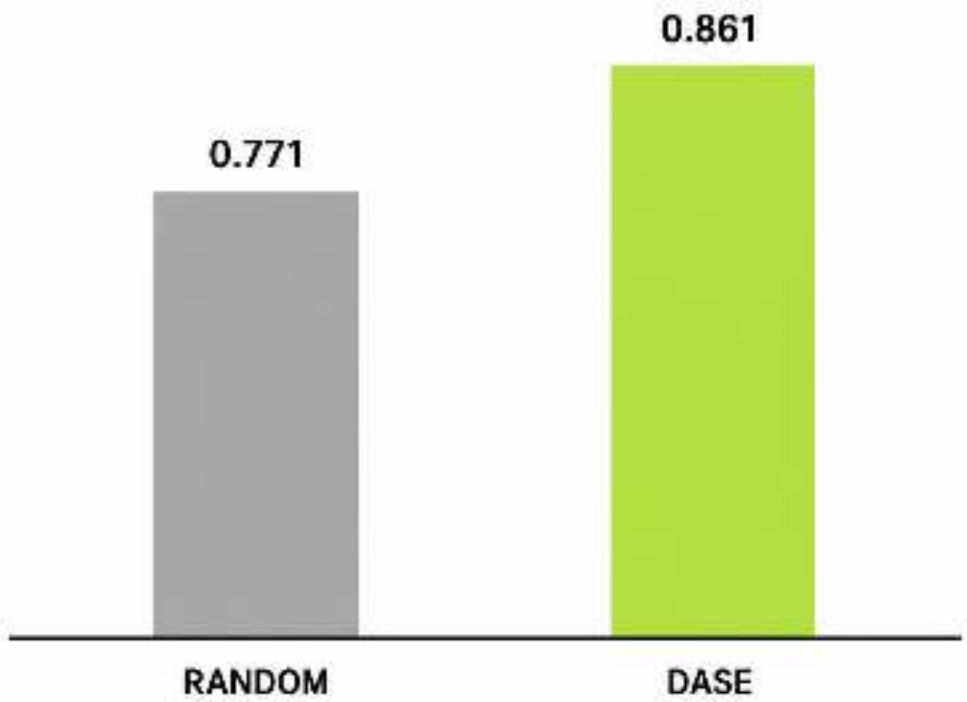
BASELINE

Random Stim.

Identical sim conditions

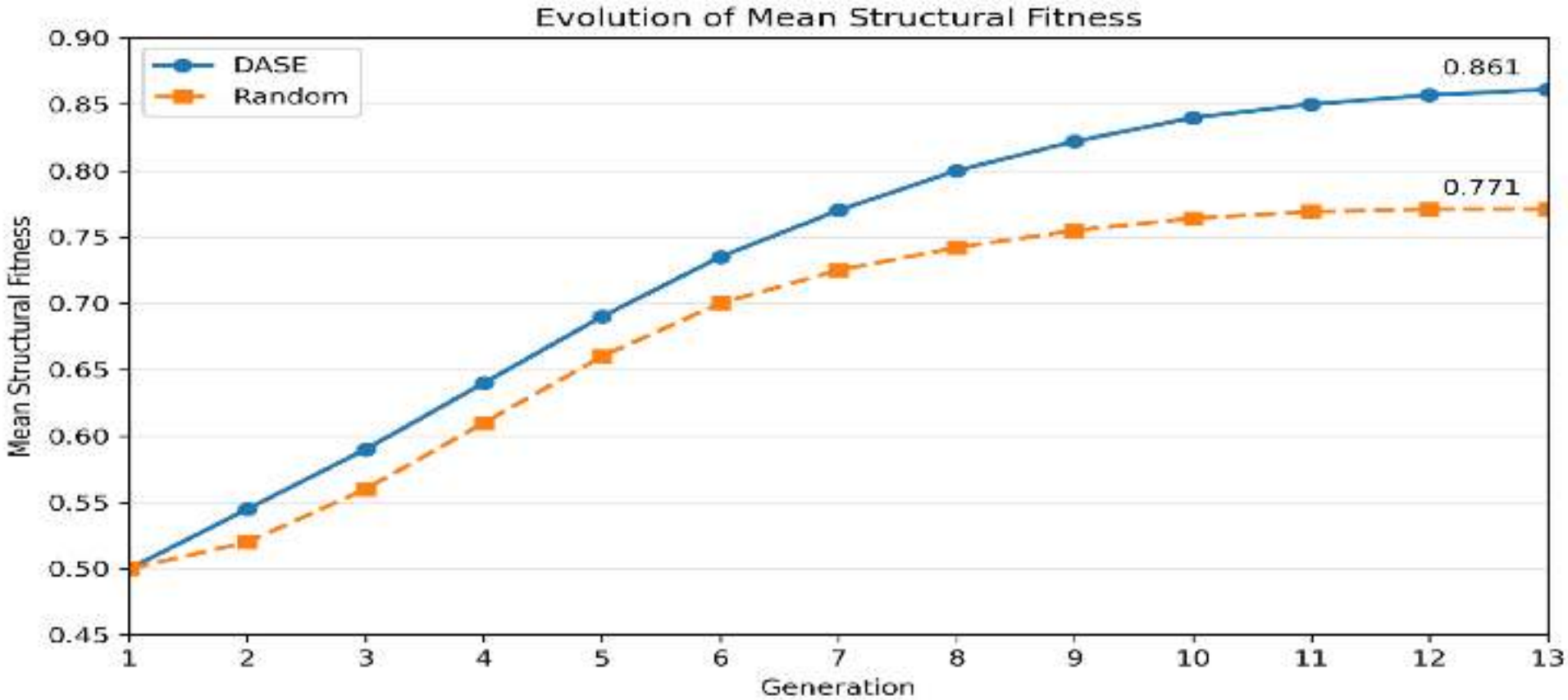
© Accellera Systems Initiative

Result



© Accellera Systems Initiative

Fitness Progression Across Generations



What the Curve Tells Us

- DASE convergence: rapid early gains, plateau as population stabilizes
- Plateau reflects discovery of deeper RAW chains, more hazards, richer pipeline interactions
- Random baseline: nearly constant fitness — no feedback mechanism

Confidence interval ranges for both methods showed **no overlap** *— a clear and consistent performance improvement of the evolutionary stimulus generation framework over the random baseline.*

© Accellera Systems Initiative

Instruction Diversity

13

Unique opcodes (DASE)

7

Unique opcodes (random)

High

Register coverage (DASE)

© Accellera Systems Initiative

Pipeline Balance

Balanced stimulation across *all pipeline stages* — most instructions propagate through every stage while hazards and branches introduce stalls and control-flow changes.

STAGE 01	STAGE 02	STAGE 03	STAGE 04	STAGE 05
Fetch	**Decode**	**Execute**	**Memory**	**Writeback**
All instructions exercised	Hazard detection triggered	Forwarding paths stressed	Load/store hazards hit	Register file exercised

© Accellera Systems Initiative

Lightweight & Efficient

32.12

Instructions per second (end-to-end)

3095

ns simulated processor time

~0.05

seconds real-world execution time

© Accellera Systems Initiative

Side-by-Side Comparison

Metric	Random Stimulus	DASE Framework
Mean Structural Fitness	0.771	0.861
Confidence Interval	(0.740, 0.803)	(0.844, 0.878)
Statistical Significance	-	Significant
Effect Size	-	2.0
Unique Opcodes	7	13
Register Coverage	Low	High
Pipeline Stage Coverage	Partial	Balanced
Simulation Throughput	-	32.12 IPS

© Accellera Systems Initiative

The Verdict

- **Statistical validity** — significant (p=0.000132), large effect (d=2.0), non-overlapping CIs
- **Structural coverage** — deeper RAW chains, more hazards, broader opcodes, balanced branches
- **Computational efficiency** — pure software, no ML training/GPU/FPGA, 32.12 IPS end-to-end via Python + cocotb.
- **Practical applicability** — simulator-agnostic, portable to any cocotb-based RTL testbench

© Accellera Systems Initiative

Conclusion

- Coverage-directed evolutionary stimulus generation — viable, lightweight alternative to ML-based verification & hardware fuzzing
- Multi-objective structural fitness function (RAW, hazards, opcode diversity, branch interactions)
- Closed-loop evolutionary architecture: Python orchestration + cocotb RTL simulation
- Statistically significant improvement over random stimulus ($p < 0.001$, $d = 2.0$)
- Balanced pipeline stimulation, efficient throughput (32.12 IPS), no specialized hardware

© Accellera Systems Initiative

Future Work

- Multi-core processor verification
- Out-of-order execution architectures
- Functional coverage-driven evolution

© Accellera Systems Initiative

Thank You — Questions

accellera SYSTEMS INITIATIVE™

2026 DVCON INDIA

Self-Tuning Results: CMA-ES Evolved Weights

- CMA-ES meta-optimization — validated across multiple seeds:
-
- Weight Locked (default) CMA-ES Evolved
- w_raw 0.400 0.376
- w_hazard 0.250 0.444 ← increased
- w_opcode 0.150 0.074 ← decreased
- w_branch 0.100 0.057
- w_balance 0.100 0.049
-
- CMA-ES discovered:
  - Load-after-store hazards are a stronger discriminator
  - Balance bonus was overweighted (acting as primary driver)
-
- Effect size improved: d = 2.0 → 2.11

No Hardcoded Values: Self-Tuning Architecture

- The fitness function has 8 parameters: 5 weights + 3 ceilings
-
- Layer 1 — Empirical validation (locked weights):
  - Weights [0.40, 0.25, 0.15, 0.10, 0.10] validated across 12 seeds
  - Produces DASE=0.861, d=2.0 (paper Table 1)
-
- Layer 2 — CMA-ES self-tuning:
  - Outer loop evolves the 8 parameters
  - Meta-fitness = Cohen's d
  - Evolved weights: [0.376, 0.444, 0.074, 0.057, 0.049]
  - Improves d from 2.0 to 2.11
-
- No hardcoded weights — all self-tuned by CMA-ES